

LangGraph Study Guide

Built from a 15-slide carousel by @techyy_bandaa, corrected and filled in against the official docs as of 2026-09-25. Verified package versions: langgraph 1.2.12, langgraph-sdk 0.4.5, langgraph-checkpoint 4.2.0. Install with `pip install -U langgraph`.

Where the deck's code does not match the current API, that is called out explicitly under "mistake to avoid" rather than silently fixed.

1. What LangGraph is (deck Page 1)

LangGraph is a low-level orchestration library for building stateful, multi-actor LLM applications as graphs: nodes are steps (LLM calls, tool calls, plain functions), edges say what runs next, and a shared state object flows through and gets updated at each step. It sits alongside LangChain rather than being a simple extension of it - you can use LangGraph with or without LangChain's chat model wrappers.

```
from langgraph.graph import StateGraph, START, END
```

Mistake to avoid: treating LangGraph as "LangChain but with more steps." The deck's own comparison table is roughly right in spirit, but the real distinguishing feature is that LangGraph graphs can cycle (loop back to earlier nodes) and pause mid-run for human input - plain LangChain chains cannot do either.

Official page: <https://docs.langchain.com/oss/python/langgraph/overview>

2. Graph structure and basics (deck Page 2)

A graph is a set of nodes connected by edges, with special `START` and `END` markers for entry and exit. Execution begins at `START`, runs a node, follows an edge to the next node based on the current state, and repeats until it reaches `END`.

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(MyState)
builder.add_edge(START, "first_node")
builder.add_edge("first_node", END)
```

Mistake to avoid: the deck shows node types as if "Agent Node," "Tool Node," "Condition Node" and "Custom Node" are distinct classes you import. They are not - every node is just a plain Python callable (or LangChain runnable) registered with `add_node()`. `ToolNode` (see section 9) is the one real prebuilt exception.

Official page: <https://docs.langchain.com/oss/python/langgraph/graph-api>

3. State and reducers (deck Pages 3-4)

State is a typed object (usually a TypedDict, sometimes a Pydantic model) that every node reads from and returns partial updates to. By default, if a node returns a value for a key, that value overwrites whatever was there. To combine values instead - e.g. append a new message to a list instead of replacing it - you attach a reducer function to that key using Annotated.

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages] # merges/appends instead of overwriting
```

`add_messages` is the purpose-built reducer for chat history (it also deserializes dict messages into LangChain message objects). For any other list, a plain `operator.add` works the same way:

```
import operator
from typing import Annotated

class State(TypedDict):
    some_key: Annotated[list, operator.add]
```

Mistake to avoid: the deck lists "append," "extend," "merge_dict" and "custom reducer" as if they were named options you choose from. There is no such menu - a reducer is just any function of shape `(current_value, new_value) -> merged_value` that you attach via `Annotated`; the docs' own built-in examples are `add_messages` and `operator.add`. Forgetting a reducer entirely on a key that multiple nodes can write to causes an `INVALID_CONCURRENT_GRAPH_UPDATE` error at runtime (details in the open questions section).

Official page: <https://docs.langchain.com/oss/python/langgraph/graph-api>

4. Nodes (deck Page 5)

Nodes are added with `add_node(name, func)`. A node function takes the current state and returns a dict of the fields it wants to change - LangGraph merges that dict into the state using each key's reducer (or overwrite, by default).

```
def agent_node(state):
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

builder.add_node("agent", agent_node)
```

Mistake to avoid: the deck's own example wraps a node in `StateNode(call_llm)` - `StateNode` is not part of the current public API. Just pass the plain function straight to `add_node()`.

Official page: <https://docs.langchain.com/oss/python/langgraph/graph-api>

5. Edges and conditional routing (deck Page 6)

Normal edges (`add_edge`) always go from A to B. Conditional edges (`add_conditional_edges`) call a routing function that inspects the state and returns the name of the next node (or `END`).

```
from typing import Literal
from langgraph.graph import END

def should_continue(state) -> Literal["tools", "__end__"]:
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools"
    return END

builder.add_conditional_edges("agent", should_continue, ["tools", END])
```

Mistake to avoid: not routing every branch to `END` (or another terminating path) causes infinite loops - the deck's own "Loops" example correctly shows a retry loop but never shows the stopping condition being checked against a max-iteration count, which is the part people actually forget to add.

Official page: <https://docs.langchain.com/oss/python/langgraph/graph-api>

6. Building your first graph (deck Page 7)

The pattern is: create a `StateGraph` with your state type, add nodes, wire them with edges, then call `.compile()` before you can run it. Without `.compile()` the graph cannot be invoked - the deck gets this specific point right.

```
from langgraph.graph import StateGraph, START, END, MessagesState

def chatbot(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)
graph = builder.compile()

graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

Mistake to avoid: calling `.invoke()` on the builder instead of the compiled graph. Also note `MessagesState` is a ready-made state type with a `messages` key already wired to `add_messages` - no need to hand-roll it for a chat-style graph.

Official page: <https://docs.langchain.com/oss/python/langgraph/overview>

7. Wiring in an LLM (deck Page 8)

Any chat model can be a node - call `.invoke()` on it with the message list and return the response. `bind_tools()` attaches tool schemas so the model can request tool calls.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0.7)
llm_with_tools = llm.bind_tools(tools)

def call_model(state):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}
```

For a fuller worked example the quickstart uses `init_chat_model()` instead of a provider-specific class, which makes it easy to swap models by string name (e.g. `init_chat_model("claude-sonnet-4-6", temperature=0)`).

Mistake to avoid: the deck's `StateNode(call_llm)` line again - drop it, just register `call_llm` directly with `add_node()`. Also, hardcoding `api_key="your-api-key"` in code as the deck shows; use an environment variable instead.

Official page: <https://docs.langchain.com/oss/python/langgraph/quickstart>

8. Memory and persistence (deck Page 9)

LangGraph has two separate persistence concepts that the deck blurs together:

Concept	Scope	What it's for
Checkpoint	single thread	conversation continuity, resuming after a crash, time travel
Store	across threads	long-term facts, user preferences, shared knowledge

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "thread-1"}}
graph.invoke({"messages": [...]}, config=config)
```

For anything beyond a demo, swap `InMemorySaver` for a real backend:

```
from langgraph.checkpoint.sqlite import SqliteSaver    # local/dev
from langgraph.checkpoint.postgres import PostgresSaver  # production

with PostgresSaver.from_conn_string("postgresql://user:pass@host/db") as checkpointer:
    checkpointer.setup()
    graph = builder.compile(checkpointer=checkpointer)
```

Mistake to avoid: using InMemorySaver (or the deck's MemorySaver) and assuming state survives a restart - it does not, it is process memory only. Also, the deck's "Long-term Memory" slide (hand-rolled save_user_memory/load_user_memory functions) reinvents what the official Store API already does - reach for that instead of building your own persistence layer. Note: both MemorySaver and InMemorySaver names appear in official docs (the persistence page uses InMemorySaver; the v1 migration guide lists MemorySaver as unchanged) - we could not find a page directly reconciling whether one is an alias of the other, so check the installed version's langgraph.checkpoint.memory module if it matters to you.

Official page: <https://docs.langchain.com/oss/python/langgraph/persistence>

9. Human-in-the-loop (deck Page 10)

interrupt() pauses a node and surfaces a value to the caller; the graph resumes from exactly that point when you resupply a Command(resume=...).

```
from langgraph.types import interrupt, Command

def approval_node(state):
    approved = interrupt("Do you approve this action?")
    return {"approved": approved}

# first call - runs until the interrupt, then stops
config = {"configurable": {"thread_id": "approval-123"}}
graph.invoke({"action_details": "Transfer $500"}, config=config)

# resume with the human's answer
graph.invoke(Command(resume=True), config=config)
```

Mistake to avoid: the deck invents from langgraph.types import Interrupt (a capitalized class) and resumes with graph.invoke({"messages": [...], "resume": True}) - neither exists in the current API. The real function is lowercase interrupt(), and resuming is done by passing a Command(resume=...) object as the input, not a resume key inside your state dict. This requires a checkpointer to be compiled in, since the graph needs somewhere to persist state while paused.

Official page: <https://docs.langchain.com/oss/python/langgraph/interrupts>

10. Tool calling and ReAct loops (deck Page 11)

Tools are plain functions decorated with `@tool`; `bind_tools()` gives the model their schemas, and `ToolNode` executes whichever tool the model asked for. A conditional edge checks `message.tool_calls` to decide whether to loop back to the model or stop.

```
from langchain_core.tools import tool
from langgraph.prebuilt import ToolNode

@tool
def search(query: str) -> str:
    """Search for information."""
    ...

tools = [search]
llm_with_tools = llm.bind_tools(tools)
builder.add_node("tools", ToolNode(tools))
```

For a full ReAct-style agent without hand-assembling the graph, use the current prebuilt helper:

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="anthropic:claude-sonnet-5",
    tools=[search],
    checkpointer=InMemorySaver(),
)
agent.invoke({"messages": [{"role": "user", "content": "What's the weather in SF?"}]},
            config={"configurable": {"thread_id": "1"}})
```

Mistake to avoid: the deck's `from langgraph.prebuilt import Tool` does not match the current API - tools are defined with `@tool` from `langchain_core.tools`, and `ToolNode` (also in `langgraph.prebuilt`) is what executes them. Separately, `create_react_agent` (the older prebuilt agent helper) is deprecated as of LangGraph v1 - use `create_agent` from `langchain.agents` instead; its `prompt=` argument was also renamed to `system_prompt=`.

Official page: <https://docs.langchain.com/oss/python/langgraph/workflows-agents>

11. Multi-agent and supervisor patterns (deck Page 12)

A supervisor is an agent whose job is to route work to other agents and collect their results, rather than do the work itself. There is a prebuilt `create_supervisor` helper in the separate `langgraph-supervisor` package, but LangChain's current guidance favors building supervisors directly with tool-calling handoffs (one agent calls a "transfer to X agent" tool) for more control over what context each agent sees, rather than reaching for the prebuilt package by default.

Mistake to avoid: the deck's supervisor code is entirely conceptual (an if/elif router picking an agent by task name) with no real handoff mechanism shown. That pattern works for toy examples but does not carry state or conversation context between agents the way a real handoff tool call does - do not treat the deck's routing snippet as production-ready.

We could not find a docs.langchain.com page giving a verified minimal code snippet for tool-based handoffs at the depth we hold the rest of this guide to, so no snippet is given here - verify against the docs below before writing this pattern into real code.

Official page: <https://docs.langchain.com/oss/python/langchain/multi-agent/subagents-personal-assistant>

12. Advanced patterns: subgraphs and map-reduce (deck Page 13)

A subgraph is a compiled StateGraph used as a node in a larger graph - this lets you build and test a piece of logic independently, then plug it in.

```
sub_builder = StateGraph(SubState)
# ... add nodes/edges to sub_builder ...
sub_graph = sub_builder.compile()

main_builder.add_node("process", sub_graph)
```

For map-reduce (running the same step over a list of items in parallel, then combining results), LangGraph's mechanism is the Send API, which the v1 migration guide confirms is unchanged from prior versions. We did not fetch a page with a verified code example of Send at the same confidence level as the rest of this guide, so no snippet is given - check the graph API docs below for current usage before relying on it.

Mistake to avoid: the deck's map-reduce example (map_fn/reduce_fn over state["data"]) is a plain Python list comprehension, not LangGraph's actual parallel-fan-out mechanism - it will run sequentially, not in parallel, and will hit the same concurrent-write error described in section 14 if multiple mapped branches write to the same state key without a reducer.

Official page: <https://docs.langchain.com/oss/python/langgraph/graph-api>

13. Error handling (deck Page 14... in the deck's numbering, Page 13)

Retries and error handling are configured per node via add_node, not via a separate decorator or global Graph() call as the deck shows.

```
from langgraph.types import RetryPolicy
from langgraph.errors import NodeError
from langgraph.types import Command

def payment_error_handler(state, error: NodeError) -> Command:
    return Command(update={"status": f"compensated: {error.error}"}, goto="finalize")
```

```
builder.add_node(
    "charge_payment",
    charge_payment,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
    error_handler=payment_error_handler,
)
```

By default, `RetryPolicy` retries on most exceptions except a documented list of "not worth retrying" ones (`ValueError`, `TypeError`, `ImportError`, and similar), and for HTTP libraries like `requests/httpx` it only retries on 5xx responses. There is also a `CachePolicy` for skipping recomputation of a node given the same input:

```
from langgraph.types import CachePolicy
from langgraph.cache.memory import InMemoryCache

builder.add_node("node_name", node_function, cache_policy=CachePolicy(ttl=120))
graph = builder.compile(cache=InMemoryCache())
```

Mistake to avoid: the deck's `from langgraph.retry import retry` decorator and `Graph().add_node(..., retry=3, timeout=60)` do not match the current API at all - there is no `langgraph.retry` module and no plain `Graph()` class. Retries are `retry_policy=RetryPolicy(...)` passed to `add_node`, and there is no built-in per-node timeout parameter in what we verified; enforce timeouts yourself inside the node function if you need them.

Official page: <https://docs.langchain.com/oss/python/langgraph/use-graph-api>

14. Debugging and observability (deck's final slide, Page 14)

LangSmith is the official tracing and observability tool for LangGraph. Enabling it is an environment-variable switch, not extra code:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=<your-api-key>
export LANGSMITH_PROJECT=my-agent-project # optional, defaults to "default"
```

For tracing only specific calls instead of everything:

```
import langsmith as ls

with ls.tracing_context(enabled=True):
    agent.invoke({"messages": [{"role": "user", "content": "..."}]})
```

Mistake to avoid: nothing wrong in the deck's concepts here (visualization, traces, state inspection are all real LangSmith features), but note that tracing is opt-in via environment variables - it is not on by default just because you installed LangSmith or LangGraph.

Official page: <https://docs.langchain.com/langsmith/observability-quickstart>

What the deck promised but never delivered

The carousel's cover slide lists 15 subtopics, ending with "13 Production Deployment," "14 Use Cases & Real-world Apps," and "15 Best Practices & Tips." None of these three exist anywhere in the 15 slides - the deck's own page numbering stops at "Page 14" (Debugging & Observability), which is topic 9 on its own contents list, not topic 15. Below is what the docs actually say for the three genuinely missing topics.

Production deployment

LangGraph apps can be deployed several ways: fully managed (LangSmith Cloud - connect a GitHub repo, submit for deployment, roughly 15 minutes to go live), hybrid, standalone Agent Servers (Docker / Docker Compose / Kubernetes, without needing the LangSmith control plane, still able to send traces to LangSmith), or a fully self-hosted control plane and data plane (requires a LangSmith Helm chart and a license with the "Engine" entitlement). For a standalone self-hosted setup, storage services are bundled by default but "it's strongly recommended to use external storage services" in production rather than the bundled ones. Source: <https://docs.langchain.com/oss/python/langgraph/deploy>

Real-world use cases

The docs and this deck both point at the same practical shapes: customer support / triage agents with tool calls and human approval gates for high-risk actions, research assistants that chain search -> read -> summarize, document Q&A systems, and multi-agent research/writing pipelines with a supervisor or planner agent coordinating specialist agents. These are illustrative patterns described across the pages already cited above (workflows-agents, multi-agent), not a separate dedicated "use cases" page we found.

Best practices

Pulled directly from the verified API details and error pages fetched for this guide, in order of how often they bite:

- Always attach a reducer (Annotated[... , add_messages] or operator.add) to any state key more than one node can write to in the same step - otherwise you get INVALID_CONCURRENT_GRAPH_UPDATE (see open questions below).
- Always call .compile() before invoking, and pass a checkpointer if you need resume, human-in- the-loop, or crash recovery.
- Use PostgresSaver (or another durable backend), not InMemorySaver, once you leave local development.
- Set a retention policy for checkpoints from day one rather than after storage becomes a problem (see open questions below).
- Prefer create_agent over the deprecated create_react_agent for new ReAct-style agents.
- Turn on LangSmith tracing (LANGSMITH_TRACING=true) before you need to debug a production incident, not during one.

The deck's four open questions, answered from the docs

1. How do I handle complex state conflicts in a large-scale multi-agent system?

The mechanism the docs give is the reducer: when two or more nodes write to the same state key in the same super-step, LangGraph raises `INVALID_CONCURRENT_GRAPH_UPDATE` unless that key has a reducer telling it how to combine the writes. The fix, verbatim from the official error page:

```
import operator
from typing import Annotated

class State(TypedDict):
    some_key: Annotated[list, operator.add] # "makes this append-only"
```

The honest limit: a reducer prevents the crash but does not remove all ambiguity - if the order in which concurrent writers execute matters to your logic (not just "append everything"), you get a silent order-dependence between writers in the same step, which is easy to miss in testing. There is no dedicated multi-agent "conflict resolution" API beyond per-key reducers; for genuinely independent agents, the more common design is to give each agent its own state key/namespace (or its own subgraph) so they simply do not write to the same key at all, rather than trying to merge conflicting writes. Source:

https://docs.langchain.com/oss/python/langgraph/errors/INVALID_CONCURRENT_GRAPH_UPDATE

2. What are the cost implications of using LangGraph's persistence features in production?

The docs state plainly: "Over long conversations, checkpoints accumulate. This can increase latency and storage costs." That is the extent of what the official persistence page says on cost - it does not give numbers, pricing tiers, or a per-checkpoint size estimate. The honest current answer: LangGraph itself is open source and checkpointing cost is just your storage backend's cost (Postgres/SQLite disk, or LangSmith Cloud's managed storage if you use that) - the docs do not publish a cost model beyond "it grows and you should manage it" (see question 4). There is a separate LangSmith Cloud pricing page for the managed hosting option, but we did not fetch it and are not guessing at numbers here. Source:

<https://docs.langchain.com/oss/python/langgraph/persistence>

3. How do custom state reducers handle concurrent node updates?

Same underlying mechanism as question 1: a reducer is called to merge the values returned by whichever nodes wrote to that key in the same super-step. For the built-in examples (`operator.add`, `add_messages`) this is a simple append, so the "custom" part is only about writing your own merge function with the same shape. What the docs do not spell out is a guaranteed write order for concurrent nodes - if your custom reducer is not commutative (e.g. it does more than append, such as picking a "winner"), the result can depend on execution order that is not part of the public API contract. The honest answer where the docs stay silent: do not write a custom reducer whose result depends on the order two concurrent nodes' writes arrive in, unless you have independently verified that order for your version - treat concurrent writers as unordered. Source:

https://docs.langchain.com/oss/python/langgraph/errors/INVALID_CONCURRENT_GRAPH_UPDATE,
<https://docs.langchain.com/oss/python/langgraph/graph-api>

4. What do deep checkpoint retention histories cost, and how do you prune them?

The persistence page directly acknowledges the cost ("increase latency and storage costs") and gives a retention strategy in plain language: "prune old checkpoints periodically or set a retention policy," suggesting "adding a cron job to delete checkpoints older than N days." It does not, however, document a single built-in "prune" method as part of the core public API in the page we fetched. The honest current

answer: there is no first-class, documented `checkpointeer.prune()` call confirmed on the official persistence page itself - the practical path described there is to run your own scheduled deletion against the checkpoint store (e.g. a `SQL DELETE FROM checkpoints WHERE created_at < ...` against Postgres, or the equivalent for whichever backend you use). Treat any specific method name you see for this (for example `delete_checkpoints/prune` on `SqliteSaver`, or `delete_thread`) as something to verify against your installed version's source before relying on it - we did not independently confirm those method signatures on a fetched docs.langchain.com or reference.langchain.com page. Source: <https://docs.langchain.com/oss/python/langgraph/persistence>

One-week path: zero to a real project

Day	Read	Build
1	Overview + Quickstart	Run the Hello World graph (section 6) locally; get <code>.invoke()</code> working
2	Graph API page (nodes, edges, state, reducers)	Add a second node and a conditional edge; deliberately trigger and fix an <code>INVALID_CONCURRENT_GRAPH_UPDATE</code> to see the reducer fix firsthand
3	Workflows and agents page	Add a real tool with <code>@tool</code> , bind it, add a <code>ToolNode</code> , wire the <code>should-continue</code> conditional edge for a ReAct loop
4	Persistence page	Swap <code>InMemorySaver</code> for <code>SqliteSaver</code> ; run the same conversation across two separate script invocations using the same <code>thread_id</code> and confirm it resumes
5	Interrupts page	Add one <code>interrupt()</code> call before a "risky" tool (e.g. anything that spends money or sends a message); practice resuming with <code>Command(resume=...)</code>
6	<code>langchain.agents</code> <code>create_agent</code> page + a chosen multi-agent page	Rebuild day 3's agent using <code>create_agent</code> instead of hand-wiring; then add a second specialized agent and a simple router between them
7	Deploy page + Observability quickstart	Turn on <code>LANGSMITH_TRACING</code> , look at a real trace of everything built so far; write down what you'd need to deploy it (checkpointer backend, retention policy, hosting choice)

By the end of day 7 you have a tool-using, persistent, resumable, traced agent - enough of a base to start a real project (the deck's own multi-agent research/writer example, or a support triage agent with a human-approval gate, are both reasonable first projects).